VPN-LAN Containerization — On-Demand Reflector + ADB-Server Boot (Phase 10)

Revision: 1 **Last modified:** 2026-07-01T16:31:40Z **Status:** Design — nothing deployed (remote deployment operator-gated §11.4.122); the local availability probe `../../../tests/vpn_lan/container_boot.sh` honest-SKIPs until the svord bridge is up and the containers primitives are present (§11.4.3) **Authority:** Inherits `constitution/Constitution.md` per §11.4.35. Phase 10 of `PLAN.md` §5 (line 143) — how the VPN-LAN helper services are packaged and booted **only** through the `submodules/containers` orchestration layer (§11.4.76), rootless Podman (§11.4.161), config-injected (§11.4.28). **Feature workstream:** `feature/vpn-aware-dynamic-routing` (§11.4.167) **Companion:** plan `PLAN.md` §2 routing map + §5 Phase 5/7/10 · the service this containerizes is designed in `reflector_design.md` §3.4 · the ADB-over-VPN consumer is `PLAN.md` §5 Phase 7 · the project-side service declaration is `vpn_lan_containers.yaml` · local proof: `../../../tests/vpn_lan/container_boot.sh`

---

1. Scope + what this phase delivers (FACT, §11.4.6)

Phase 10 packages the two long-running helper services the VPN-LAN feature needs on the **remote (device-side)** subnet so helix_proxy-side clients can use them over the routed L3 VPN:

Service	Why it must run on the remote subnet	Designed in
Discovery reflector (Avahi `enable-reflector` for mDNS + an SSDP/WS-Discovery relay)	multicast discovery does not cross an L3 VPN — a reflector must sit where `224.0.0.251` / `239.255.255.250` are locally visible (`reflector_design.md` §2/§3)	Phase 5
ADB server helper (a central `adb server`	one adb server fronts many remote devices, so helix_proxy-side `adb connect 10.x:5555` needs	Phase 7

Service	Why it must run on the remote subnet	Designed in
reachable over routed TCP 5555)	no proxy hop (PLAN.md §5 Phase 7 T7.1/T7.2)	

This phase delivers, autonomously, exactly three artifacts — a design (this document), a project-side service declaration (vpn_lan_containers.yaml), and a local availability probe (container_boot.sh) that honest-SKIPs when it cannot prove readiness. It does **NOT** boot any container and does **NOT** deploy anything on any remote host — remote deployment changes a remote host and is therefore **operator-gated** (§11.4.122, §7 below). The honest boundary is §7.

---

2. Orchestration layer — `submodules/containers` only (§11.4.76)

Every containerized VPN-LAN workload boots **on-demand** through the `submodules/containers` primitives — **never** an ad-hoc `podman`/`docker` invocation in this repo (Hard Stop #3 / §11.4.76 / §11.4.161). The three primitives Phase 10 composes, all present in the submodule (FACT — verified on disk):

- **`pkg/compose`** — the service model + compose orchestration. `HelixService` (name, image, ports, env, volumes, `DependsOn`, `HealthCheck`, `ResourceLimits`) grouped into a `HelixComposeProject`; the `ComposeOrchestrator` (`Up`/`Down`/`Status`/`Logs`) shells out to the detected compose provider. On this host the detected provider is **`podman compose`** (the podman CLI's docker-compose-compatible provider — rootless, §11.4.161).
- **`pkg/boot`** — the high-level `BootManager` that registers endpoints (name + compose file + health check), brings each service up via the orchestrator, and gates readiness on the health checker. Constructed with functional options (`WithOrchestrator`, `WithHealthChecker`, `WithRuntime`, `WithProjectDir`, …).
- **`pkg/health`** — the `HealthChecker` (TCP / HTTP / gRPC / custom, with retry) that decides a service is genuinely up. The reflector's readiness is a **TCP** probe on the SSDP/mDNS relay ports; the adb-server's is a **TCP** probe on 5555.

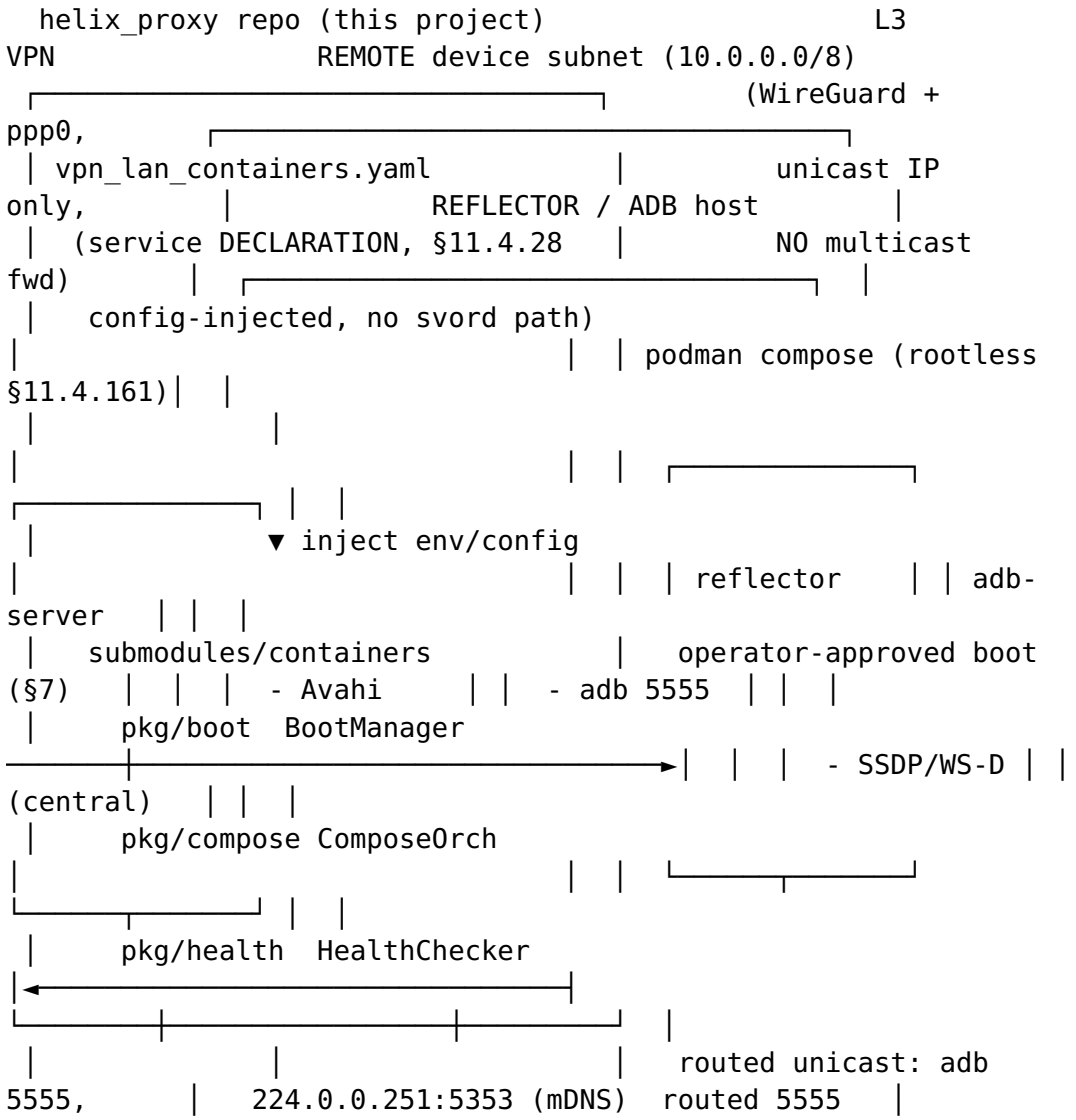
The `cmd/boot` CLI is the operator-facing entry point that composes the three; the project never re-implements orchestration (§11.4.74).

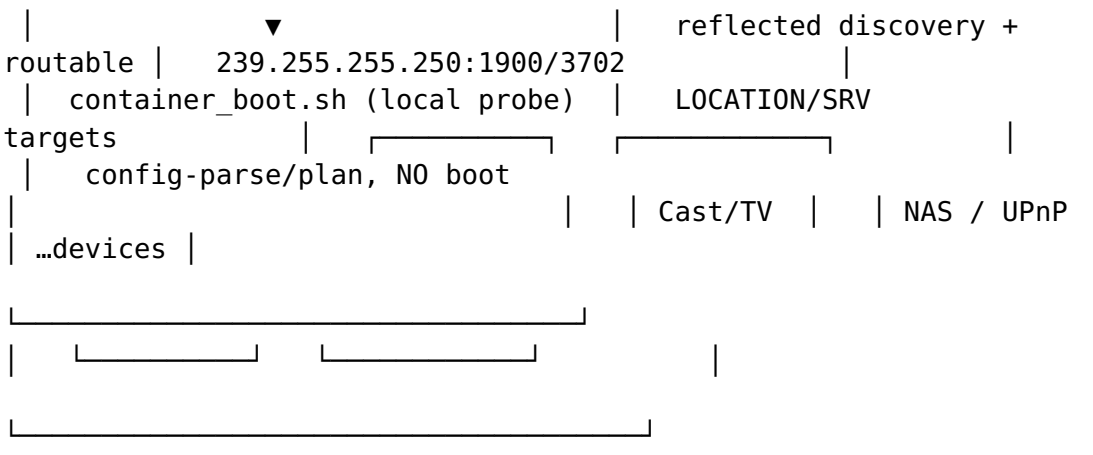
2.1 The on-demand-infra invariant (§11.4.76)

The reflector + adb-server are **not** started by hand. Boot is part of the **test/deploy entry point**: the entry point asks `pkg/boot` to bring the declared services up (once the operator has approved the remote host, §7), and the operator is **never** required to run `podman` / `podman compose up` manually. Symmetrically, an integration test that claims to exercise the reflector MUST actually boot it via the submodule — a short-circuit fake that skips boot is a §11.4 violation. This local phase honours the invariant by making its readiness probe (`container_boot.sh`) the single entry point that validates the boot plan (config-parse / plan level) without starting a container.

---

3. Topology — project → containers-submodule boot → remote-subnet containers





The **declaration** (`vpn_lan_containers.yaml`) lives in the project; the **orchestration engine** lives in `submodules/containers`; the **containers** run on the operator-designated remote host where physics lets the reflector see the local multicast groups. The project injects config INTO the submodule at boot time (§4) — it never writes project context INTO the submodule tree.

---

4. Config-injection, not coupling (§11.4.28 + §11.4.74)

The submodule stays **project-agnostic**. Everything VPN-LAN-specific is **injected** and lives in the project:

- The service **declaration** (`vpn_lan_containers.yaml`) is a project file, **NOT** a file inside `submodules/containers`. The submodule consumes it as a `ComposeProject.File` (or the equivalent `HelixService` set) — it carries **no** helix_proxy hardcoding.
- Deployment-time values — the remote subnet, the allowed reflector interfaces, the SSDP `LOCATION`-rewrite target, the adb-server bind address — are supplied as **environment / config injected into the container** (the `${VPN_LAN_*}` placeholders in the YAML), resolved from the project's gitignored `.env` (per the `PLAN.md` §3 bridge contract). **No svord path, no secret, and no project-specific literal ever lands inside the submodule** (§11.4.28, §11.4.10 — names/paths only).
- **No nested own-org submodule** (§11.4.28(C)): `submodules/containers` is reached from the project root; it does not pull the reflector image definitions in as its own nested submodule.
- If the submodule cannot yet model a capability (e.g. the SSDP relay is not an image it can boot, or a rootless multicast/host-network mode is unsupported), the fix is to **extend `submodules/containers` upstream** (§11.4.74 extend-don't-reimplement) — **never** a raw `podman` command worked around in this repo, and never a fork of the orchestration logic here.

---

5. Readiness verification — the local availability probe (autonomous slice)

container_boot.sh proves the boot **plan** is ready without starting a container, with the exact anti-bluff discipline of every VPN-LAN test (it mirrors discovery_reflect.sh):

- **Bridge gate first (§11.4.3 / §11.4.69):** sources `tests/lib/svord_bridge.sh`, calls `bridge_require`; svord bridge **down/misconfigured** (the default autonomous state, no `.env`) ⇒ honest SKIP + exit 0 — the path that runs now. No container is planned or contacted.
- **Scored check (bridge up) — a config-parse / plan-level readiness proof, NEVER a boot:** asserts the containers-submodule boot primitive is **invocable for the declared services** — (a) `pkg/boot` + `pkg/compose` + `pkg/health` are present in the submodule, (b) a rootless container runtime (`podman`) is on `PATH` (presence-checked, **never invoked** — the probe never runs `podman`), and (c) the project-side vpn_lan_containers.yaml **parses** and declares both services (`vpn-lan-reflector` + `vpn-lan-adb-server`) with the required fields (image, ports, healthcheck) and only injected (`${…}`) config — a config-parse / plan check that starts nothing.
- **Honest SKIP, never a fake PASS:** containers submodule primitives absent ⇒ `topology_unsupported`; `podman` absent ⇒ `hardware_not_present`; no YAML parser (`python3`) available ⇒ `topology_unsupported`; bridge down ⇒ the gate's `network_unreachable_external`.
- **Fail-closed:** a genuinely **malformed** declaration (unparseable YAML, or a missing required service/field) ⇒ **FAIL** — a broken deliverable is never SKIPped away.
- **Never boots, never kills:** the probe starts no container, runs no `podman`/`docker`, issues no `pkill`/`kill`, and never touches the data-plane proxy config or Squid (invocation-only, §11.4.122 / §11.4.119).

The PASS evidence is a captured `readiness.evidence` file under `qa-results/vpn_lan/phase10/<UTC-ts>/` enumerating the present primitives, the runtime, and the parsed service set (§11.4.5 / §11.4.69).

---

6. Resource + safety posture

- **Rootless Podman only** (§11.4.161) — no `sudo`, no rootful Docker; the runtime is presence-checked, never invoked, by the local probe.

- **Resource limits** are declared per service (`ResourceLimits` / compose `deploy.resources`) so a booted reflector stays bounded (§12.6 host-memory posture carries to the remote host too).
- **No host-power operations** anywhere in the boot path (CONST-033).
- **No data-plane contact** — the VPN-LAN helper services are additive; the boot path never touches the base proxy config, Squid, Dante, or `:53128` (§11.4.119 single-resource-owner).

---

7. Honest boundary — remote deployment is operator-gated (§11.4.122 / §11.4.6)

Booting these containers on a remote host CHANGES that host — it starts daemons, joins multicast groups (reflector), and opens an adb control surface (adb-server). Under §11.4.122 (no silent change to any remote/connected host) and `PLAN.md` §1 hard-constraint 2, helix_proxy MUST NOT deploy autonomously. The sequence:

1. **Ask first (§11.4.66 interactive options), BEFORE any remote deployment** — present the operator a keep/deploy decision with the concrete blast radius (which host, which subnet, which interfaces, which ports; that daemons start and a device-control surface opens) and the teardown path (`ComposeDown`).
2. **Only on explicit operator approval** does `pkg/boot` bring the declared services up on the operator-designated remote host via `submodules/containers`.
3. **Until then nothing is deployed**, and `container_boot.sh` validates the **plan** only (§5) and honest-SKIPs the live paths (§11.4.3).

What this design guarantees vs. does not (§11.4.6): it guarantees a **correct, standards-grounded, config-injected containerization design** and a **local probe that proves the boot plan is well-formed or honest-SKIPs** — it does **NOT** guarantee a live reflector/adb-server until the operator approves and the services are actually booted on the remote subnet (§7 step 2). No phase is "done" until its runtime signature verifies with captured evidence (§11.4.108) and it crosses independent review (§11.4.142) + the §11.4.169 test-type matrix.

Sources verified 2026-07-01

- **`submodules/containers pkg/compose`** — `HelixService` / `HelixComposeProject` service model + `ComposeOrchestrator` (`Up`/`Down`/`Status`/`Logs`) + `detectComposeCmd` (docker compose / docker-compose / podman-compose / **podman compose** providers): on-disk FACT, `submodules/containers/pkg/compose/{helix_project.go,orchestrator.go,types.go}`.
- **`submodules/containers pkg/boot`** — `BootManager` + functional options (`WithOrchestrator` / `WithHealthChecker` / `WithRuntime` / `WithProjectDir`): on-disk FACT, `submodules/containers/pkg/boot/{manager.go,options.go}`.
- **`submodules/containers pkg/health`** — `HealthChecker` (TCP / HTTP / gRPC / custom + retry): on-disk FACT, `submodules/containers/pkg/health/{checker.go,tcp.go,http.go,grpc.go,custom.go}`.
- **`submodules/containers/CLAUDE.md`** — the mandatory orchestration flow (`make build → ./bin/…`, never manual `podman compose up`), the on-demand-infra invariant, rootless runtime, and config-injection seam: on-disk FACT.
- **Avahi reflector + SSDP relay design** — the reflector this containerizes: `reflector_design.md` §3.4 (containerization) + its cited RFC 6762/6763/5771/2365 + Avahi/UPnP/WS-Discovery sources.

Access date for all sources: 2026-07-01. FACT items (the submodule package/API set, the podman-compose provider path, the rootless mandate) are grounded in the on-disk submodule code + `submodules/containers/CLAUDE.md`. The exact reflector/adb-server container images are a deployment-time choice pinned + cited when the operator approves deployment (§11.4.150 / §7), never asserted as settled here (§11.4.6). Deep-research (§11.4.150), multi-angle: orchestration-layer (the submodule API), routing-layer (why the services live on the remote subnet, `reflector_design.md` §2), safety-layer (operator-gated remote change §11.4.122) — access date 2026-07-01.